Advanced Time Series Project

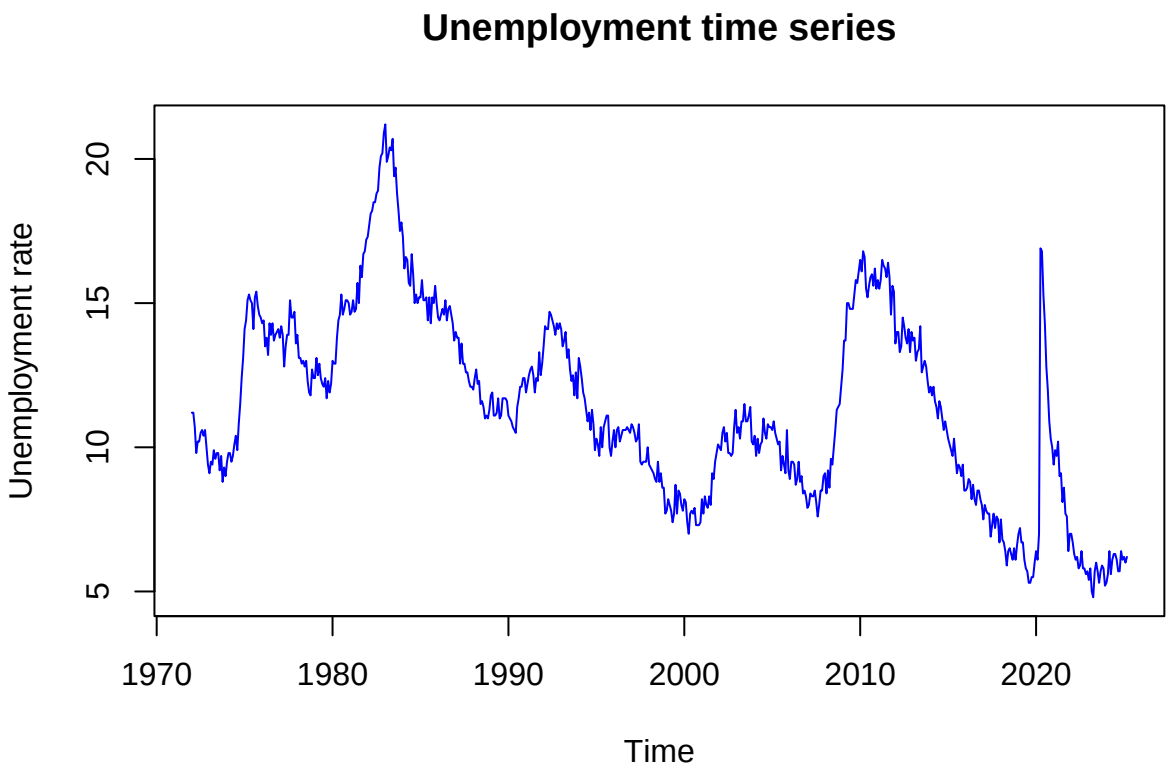
Baio Federico - Magnano Andrea - Pistone Alessio - Ravaioni Jody

2025-05-04

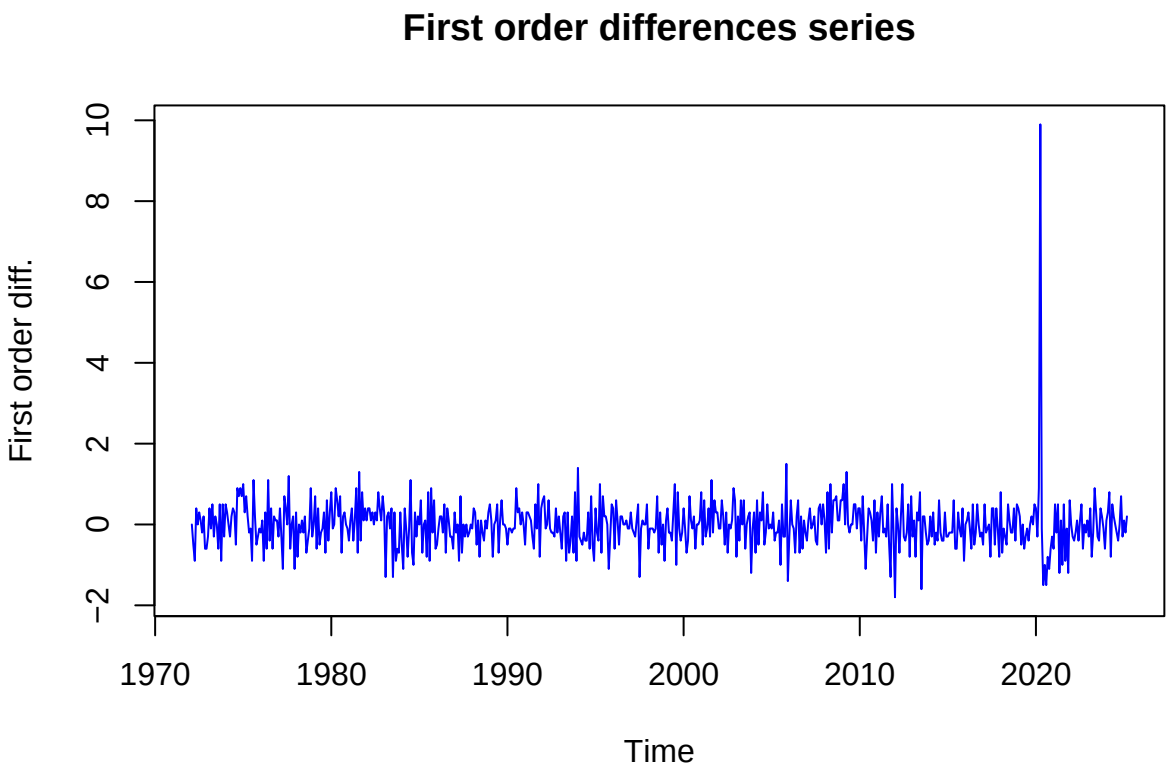
The data

The data for this analysis come from the U.S. Bureau of Economic Analysis, Personal Saving Rate [PSAVERT], retrieved from FRED, at https://fred.stlouisfed.org/series/PSAVERT, April 29, 2025. Once laoded the useful packages (stats and tseries), the first thing to do was to plot the time series.

```
df <- read.csv("LNS14000006.csv")
ts_data <- ts(df$LNS14000006, start=c(1972,1), frequency=12)

# Plot the time series and its difference
## time series
ts.plot(ts_data,xlab="Time",ylab="Unemployment rate",
        main="Unemployment time series",col="blue")
```




```r
## first order differences
ts.plot(diff(ts_data),xlab="Time",ylab="First order diff.",
        main="First order differences series",col="blue")
```



At first glance, it is clear that the *unemployment rate* was affected by the last two greatest crisis: the 2008 financial crisis and the COVID-19 one from 2020, even though the second one was more impactful in the short term (as it can be seen from the plot of the first order differences of the time series).

First approach: classical time series decomposition through moving averages smoothing

Since the variance seems to remain stable regardless the value of the series, we can try and decompose the series with an additive model. The data are already seasonally adjusted, therefore to obtain is the deterministic component.

```
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE)

ts_trend <- stats::filter(ts_data, rep(1/12, 12), sides = 2)
#smoothing the series with a finite Moving Average filter

ts_residual <- ts_data - ts_trend # should be a stationary random noise component
ts_residual <- na.omit(ts_residual)
mean(ts_residual) # the residuals shows null mean
```

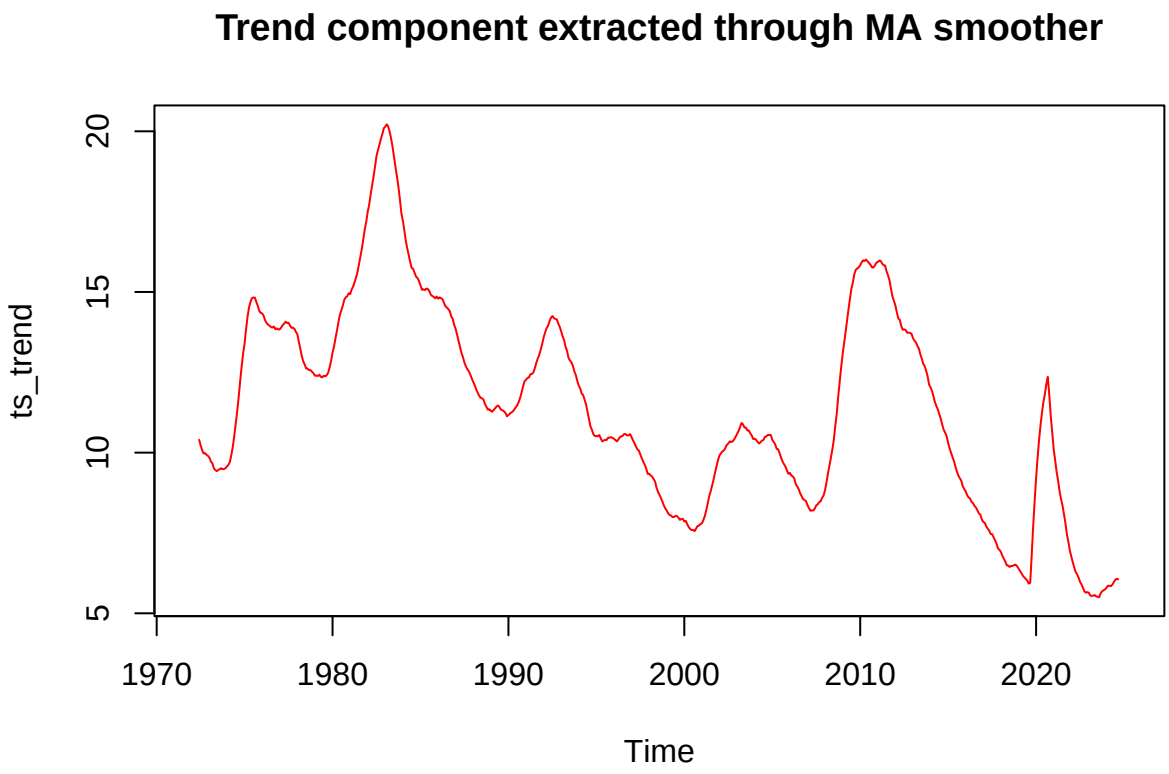
```
## [1] 0.003410297
```

```
adf.test(ts_residual)
```

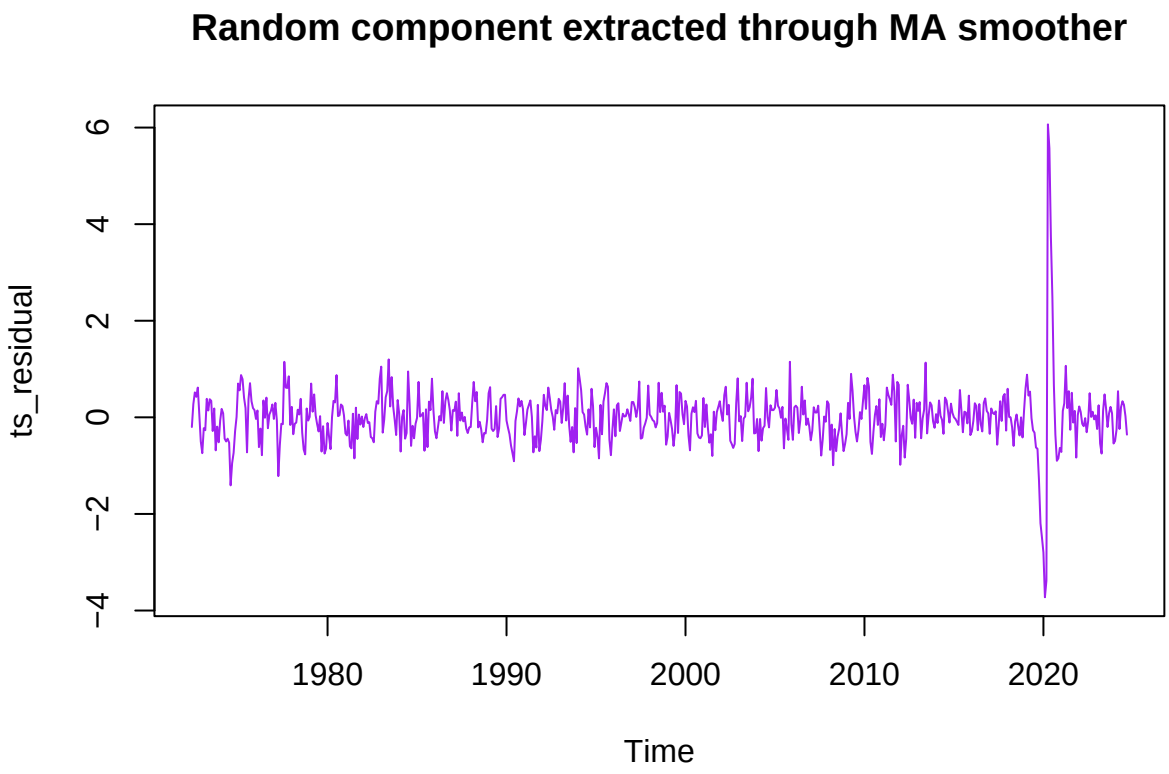
```
## Warning in adf.test(ts_residual): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  ts_residual
## Dickey-Fuller = -10.917, Lag order = 8, p-value = 0.01
## alternative hypothesis: stationary
```

```
#the Augmented Dickey Fuller test shows that this series is indeed stationary

ts.plot(ts_trend, col= "red",
        main = "Trend component extracted through MA smoother")
```




```r
ts.plot(ts_residual, col = "purple",
        main = "Random component extracted through MA smoother")
```



Thus, we extracted the trend component and the noise component from the time series.

Second approach: analysis with linear models for time-varying parameter

Since in the time series does not seems reasonable to fit a classic model, we tried to see if we could find similarities with the time-varying models that we have studied.

We firstly checked if the series was stationary by using Augmented Dickey Fuller Test for stationariety

```
adf.test(ts_data, alternative = "stationary")
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  ts_data
## Dickey-Fuller = -2.8975, Lag order = 8, p-value = 0.1984
## alternative hypothesis: stationary
```

Since the p-value is 0.2025, the series is not stationary. Therefore, we tried to differentiate it until it, eventually, became stationary.

```
diff_ts_data <- diff(ts_data)
adf.test(diff_ts_data, alternative = "stationary")
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  diff_ts_data
## Dickey-Fuller = -8.5544, Lag order = 8, p-value = 0.01
## alternative hypothesis: stationary
```

The first order differences seemed to be stationary given that the p-value of the Augmented Dickey Fuller Test is equal to 0.01.

SF1 unconditional moments, ACF and PACF of the time series

Then, we moved to compute the SF1 unconditional moments of the time series.

```
mean.series <- mean(ts_data)
var.series <- var(ts_data)
skewness.series <- mean(((ts_data-mean.series)/(var.series^0.5))^3)
kurtosis.series <- mean((ts_data-mean.series)^4)/(var.series^2)
```

```
## mean: 11.37418
```

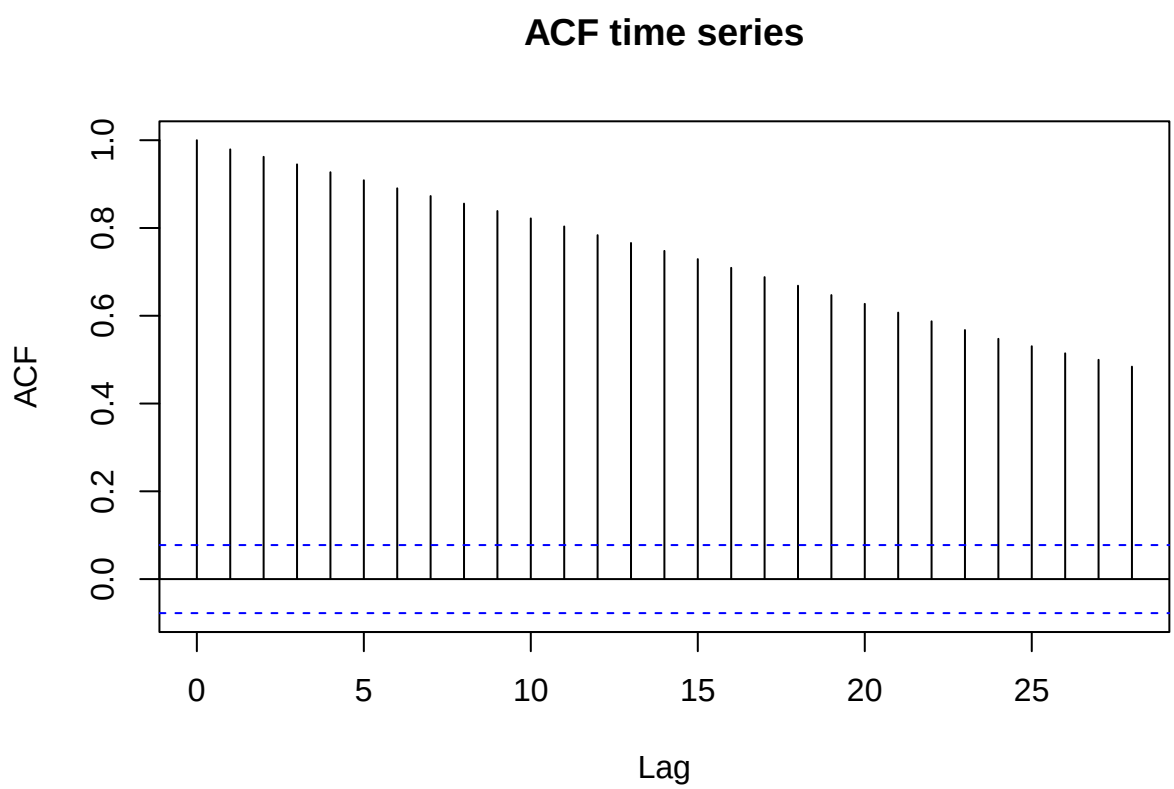
```
## variance: 11.02881
```

```
## skewness: 0.2822598
```

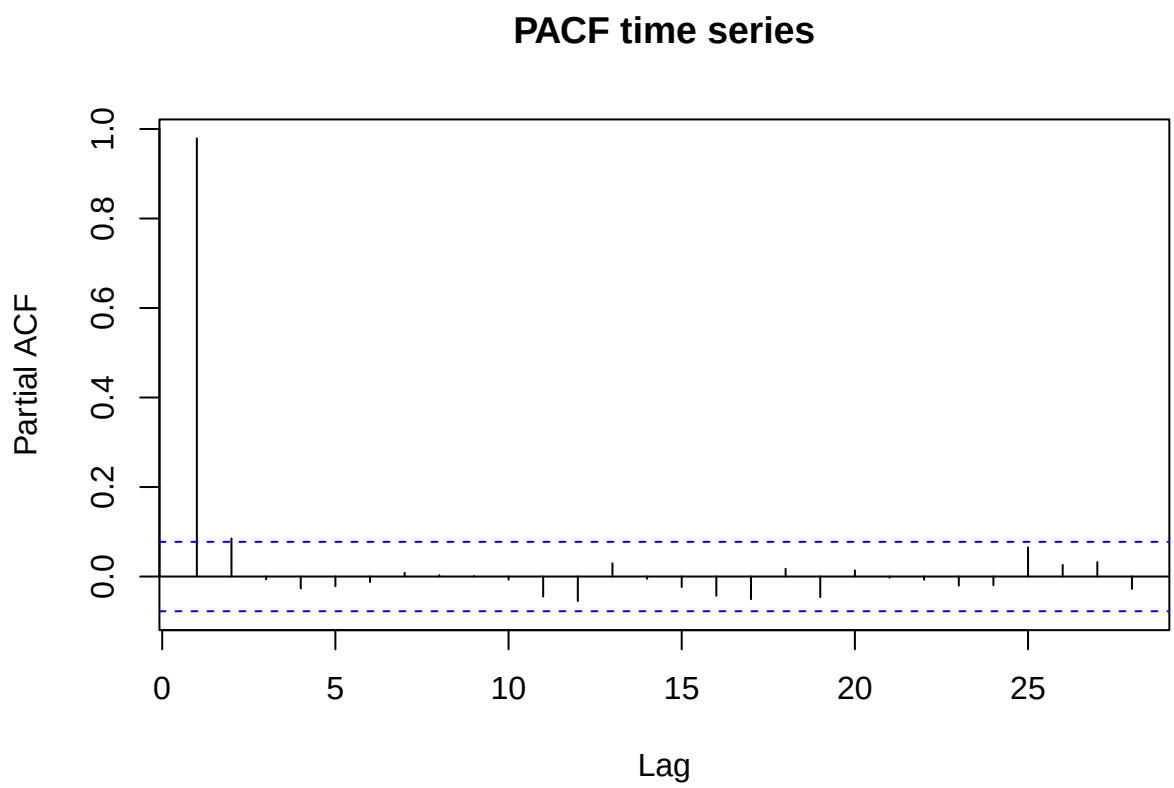
```
## kurtosis: 2.674078
```

Obtained the unconditional moments it remained to see the behaviour of the autocorrelation (ACF) and partial autocorrelation function (PACF).

```r
acf(as.vector(ts_data),main="ACF time series")
```

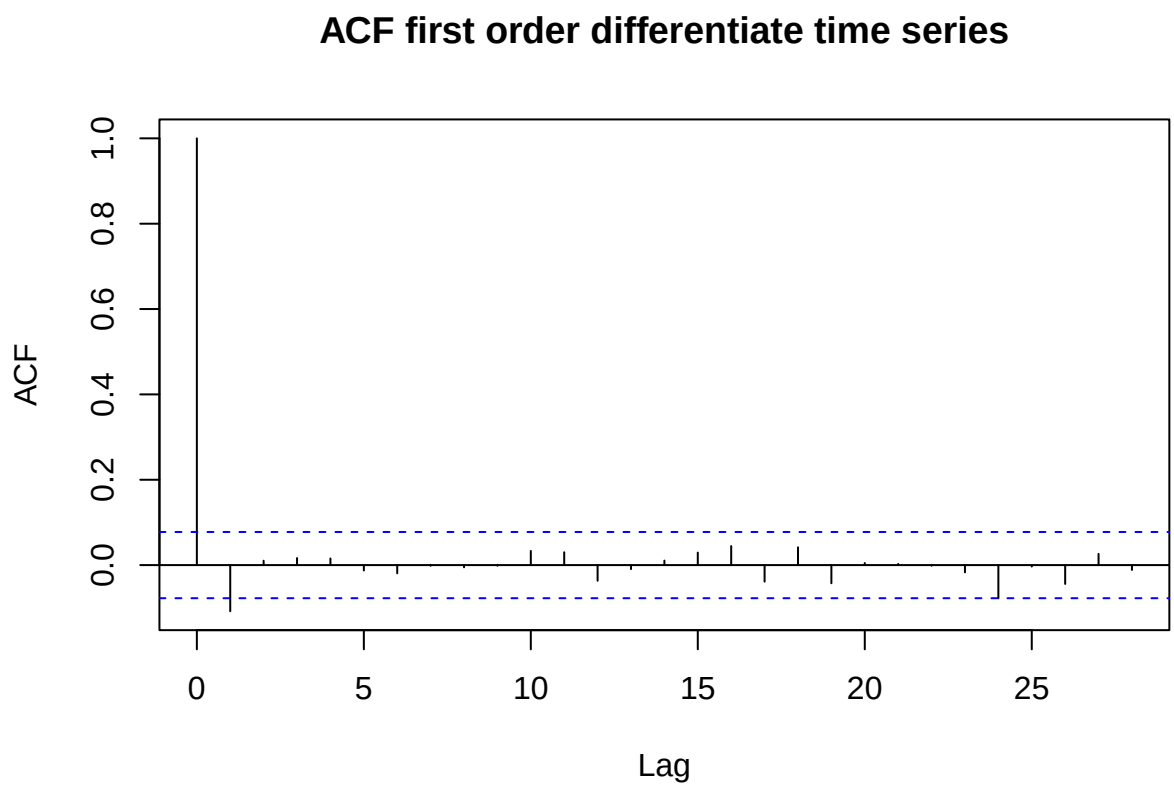



```r
pacf(as.vector(ts_data),main="PACF time series")
```



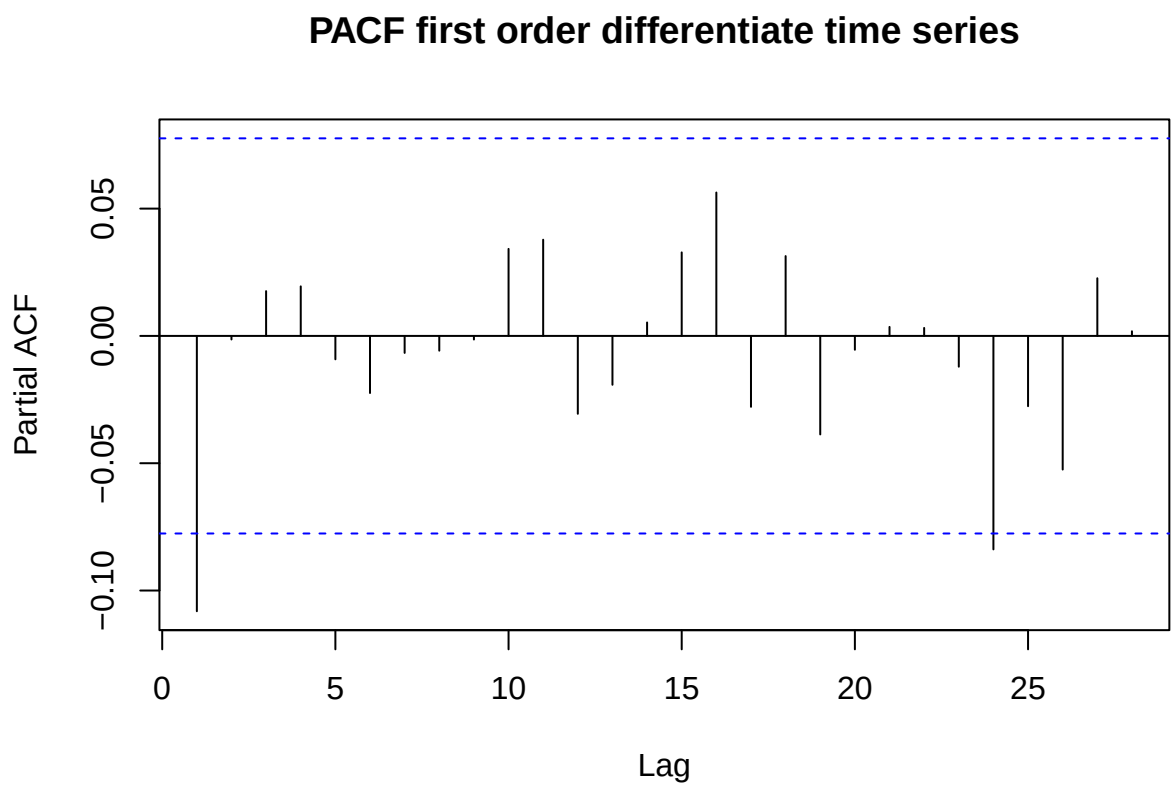
Since the ACF was decreasing exponentially and the PACF had a significative peak at lag 1, the process seems to replicate an AR(1). However, since we saw that the process was not stationary it seemed reasonable to check the ACF and the PACF of the once differentiated time series, which is indeed stationary,looked like an MA(1), exactly as the theory of an LLM (Local Level Model) suggests.

```
acf(as.vector(diff_ts_data),main="ACF first order differentiate time series")
```



After lag 2 the fluctuations of the ACF do not exceed the significance bounds,that suggest that differencing has made the series stationary.

```
pacf(as.vector(diff_ts_data),main="PACF first order differentiate time series")
```



At lag 1, we observe a peak that exceeds the significance bounds, suggesting an AR(1) model for the differenced series. Since the subsequent lags do not exceed the bounds, this supports an AR(1) structure rather than a more complex AR(p) model with p > 1.

Local Linear Model (LLM)

A Local Level Model (LLM) is a model of the type:

$$y_t = \mu_t + \epsilon_t,$$

where:

- $\mu_{t+1} = \mu_t + \eta_t$, $\eta_t \sim NID(0, \sigma_\eta^2)$,
- $\epsilon_t \sim NID(0, \sigma_\epsilon^2)$,
- $\epsilon_t \perp \eta_t \ \forall t, s$,

The reduced form of Δy_t=$y_t - y_{t-1}$ is:

$$\Delta y_t = \eta_{t-1} + \epsilon_t - \epsilon_{t-1},$$

or, from another formulation:

$$\Delta y_t = \xi_t - \theta \xi_{t-1}, \quad \xi_t \sim NID(0, \sigma_\xi^2),$$

The model

We estimated the value of θ for the MA(1) representation of Δy_t.

```
fit_arima <- arima(ts_data,order=c(0,1,1))
theta <- fit_arima$coef
sigma2_xi <- fit_arima$sigma2
```

We estimated a value of θ for -0.1062, and a value for σ_ξ^2 of 0.4141. Theoretically, the value of θ should be included between 0 and 1 but, given our formulation, the bounds to consider are -1 and 0. Since we know that $\sigma_\xi^2 = \frac{\sigma_\eta^2}{(1+\theta^2)}$, we simply obtained σ_η^2 by solving the equation above for it, obtaining a value of 0.4206.

```
sigma2_eta <- sigma2_xi*(1 + theta^2)
```

Lastly, the variance of the error term, ϵ_t, is obtainable as $\sigma_\epsilon^2 = \frac{Var(\Delta y_t) - \sigma_\eta^2}{2}$.

```
sigma2_epsilon <- var(diff_ts_data)-sigma2_eta
```

We obtained that σ_ϵ^2 is almost equal to 0 which is correct since we observed that y_t was a non-stationary AR(1), thus it is a random walk. In fact, a LLM which has $\sigma_\epsilon^2 = 0$ implies that $y_t \sim RW$.

Forecasting with the Kalman filter

Inital conditions

To inizialize the algorithm of the Kalman filter it was mandatory to specify the initial conditions, so $\mu_{1|0}$ and $P_{1|0}$. In this specific case, we opted for $\mu_{1|0} = y_1$ and $P_{1|0} = q$, where $q = \frac{\sigma_\eta^2}{\sigma_\epsilon^2}$ is the signal-to-noise ratio. Then, $\mu_{t+1|t}$, $P_{t+1|t}$ all the other values (v, K, F) were calculated iteratively.

```
n <- length(ts_data)
y <- as.numeric(ts_data)

q <- sigma2_eta/sigma2_epsilon # signal to noise ratio

mu_pred <- numeric(n)  # mu_{t|t-1}
P <- numeric(n)
v <- numeric(n)
K <- numeric(n)
F <- numeric(n)
llk <- numeric(n)

mu_pred[1] <- y[1]
P[1] <- q
```

The SSF for a Local Level Model Specifically for a LLM, here are listed the values of the unobserved components and time-varying parameters and the values of the system matrices.

- $Z_t = 1$ (representation matrix),
- $\alpha_t = \mu_t$,
- $G_t = \sigma_\epsilon$ (scale matrix),
- $T_t = 1$ (transition matrix),
- $H_t = \sigma_\eta$,
- $Q_t = 1$,

The iteration

The Kalman filter algorithm iteratively calculate the prediction of some quantities at time t+1 given the information at time t. It simply works as follows:

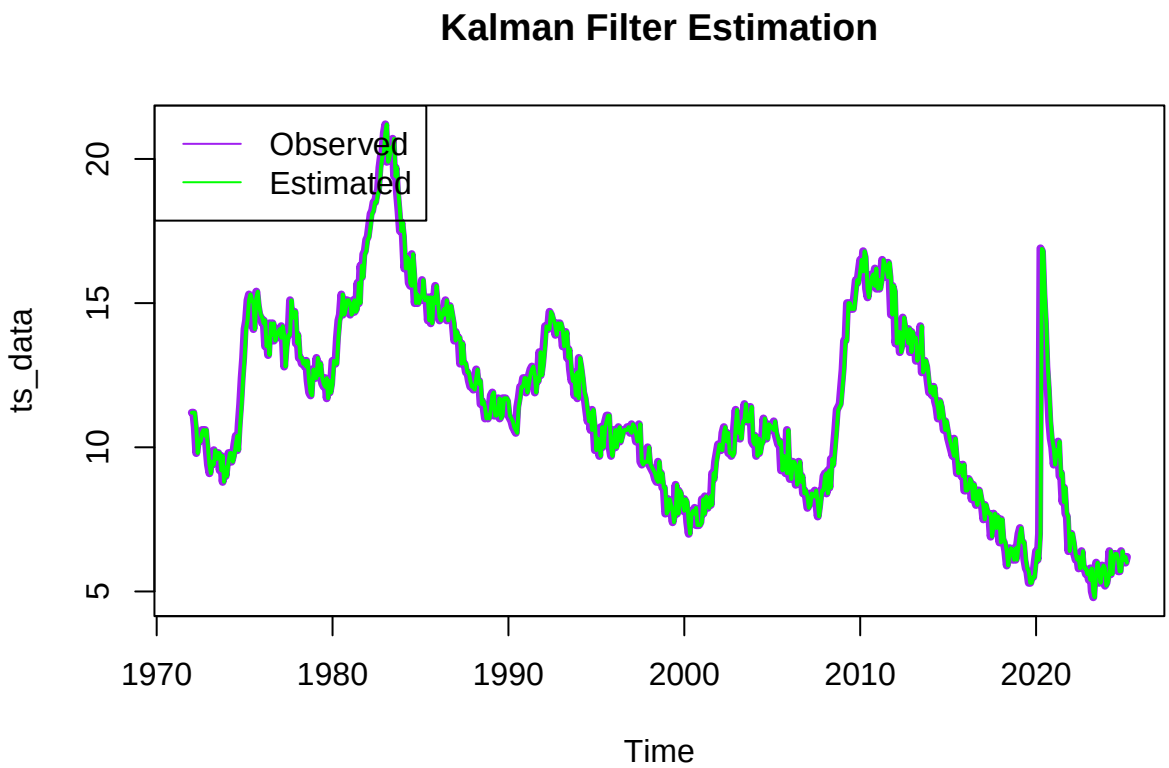
- given a prediction $y_{t|t-1}$, once the time t arrives, y_t is observed and the prediction error $\upsilon_t = y_t - y_{t|t-1}$ is computed;
- then, the innovation variance $F_t = P_t + \sigma_\epsilon^2$ is calculated;

- the Kalman gain is computed as $K_t = \frac{P_t}{F_t}$;
- the new prediction for y_{t+1} is $y_{t+1} = y_t + K_t \upsilon_t$;
- and the variance of the prediction, $P_{t+1|t} = P_t + \sigma_\eta^2 - K_t F_t K_t$.

```
for (t in 1:(n-1)) {
  # Prediction error
  v[t]<-y[t]-mu_pred[t]
  # Innovation variance
  F[t]<-P[t]+sigma2_epsilon
  # Kalman gain
  K[t]<-P[t]/F[t]
  # Update step
  mu_pred[t+1]<-mu_pred[t]+K[t]*v[t]
  P[t+1]<-P[t]+sigma2_eta-K[t]*F[t]*K[t]
  # Log-likelihood
  llk[t]<--0.5+log(F[t]+v[t]^2/F[t])
}

loglik <- sum(llk)
```

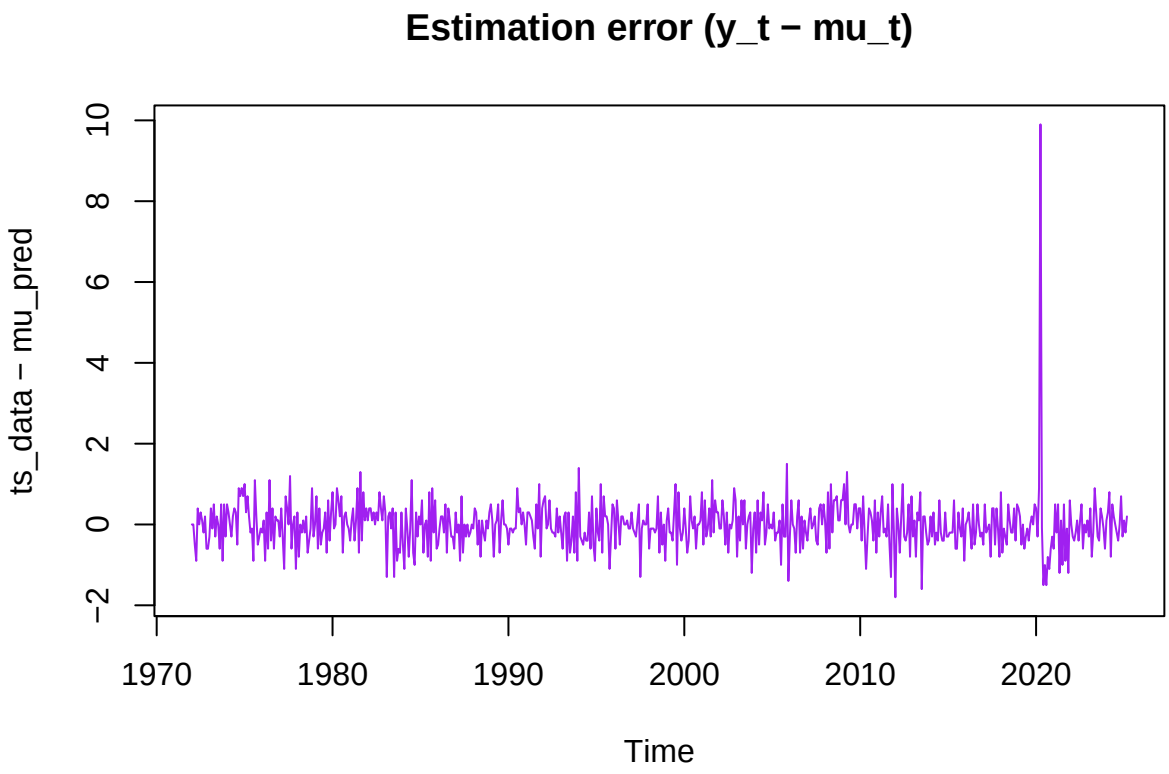
Finally, we plotted the time series and the results from the Kalman filter. In these graphical representations is clearly shown that the algorithm performed almost perfectly with respect to this time series.

```
ts.plot(ts_data, main = "Kalman Filter Estimation",
        col = "purple",type = "l", lwd = 4)
lines(ts(mu_pred, start = start(ts_data), frequency = frequency(ts_data)),
      col = "green",lwd=2)
legend("topleft", legend = c("Observed", "Estimated"),
       col = c("purple", "green"), lty = 1)
```



The Kalman filter captures the signal very well, as evidenced by the overlap of the two curves. This is due to the fact that, as shown above, the value of the variance of ϵ is almost zero, and therefore predicting μ, we can almost perfectly follow the trend of our series

```
ts.plot(ts_data - mu_pred, col = "purple", main = "Estimation error (y_t - mu_t)")
```



The estimation error fluctuates around zero, indicating that overestimations and underestimations are generally balanced. At certain points the peaks significantly exceed the confidence bounds, suggesting that the model is not so able to capture steep changes in the data.

The role of q: signal-to-noise ratio

The quantity q, called the signal-to-noise ratio, is defined as the ratio between the signal variance σ_η^2 and the noise variance σ_ϵ^2:

$$q = \frac{\sigma_\eta^2}{\sigma_\epsilon^2}.$$

If q tends to zero, the noise variance dominates the signal variance; conversely, if q diverges, the signal variance dominates the noise variance. The value of q affects the ability of the Kalman Filter to capture the underlying signal. When q is small, the Kalman Filter assigns more weight to the predictive model rather than to the observations.

The Kalman Filter and the parameter q are linked through the matrix P, since the Kalman Gain at time t is defined as:

$$K_t = \frac{P_t}{F_t} = \frac{P_t}{P_t + \sigma_\epsilon^2},$$

and the evolution of the prediction error variance is given by:

$$P_{t+1|t} = P_t + \sigma_\eta^2 - K_t^2 F_t.$$

We need an initial value for P, which is typically set as:

$$P_1 = q = \frac{\sigma_\eta^2}{\sigma_\epsilon^2}.$$

Now let's investigate how the Kalman Filter empirically behaves as a function of q, by defining hypothetical values for the two variances that are not related to our time series:

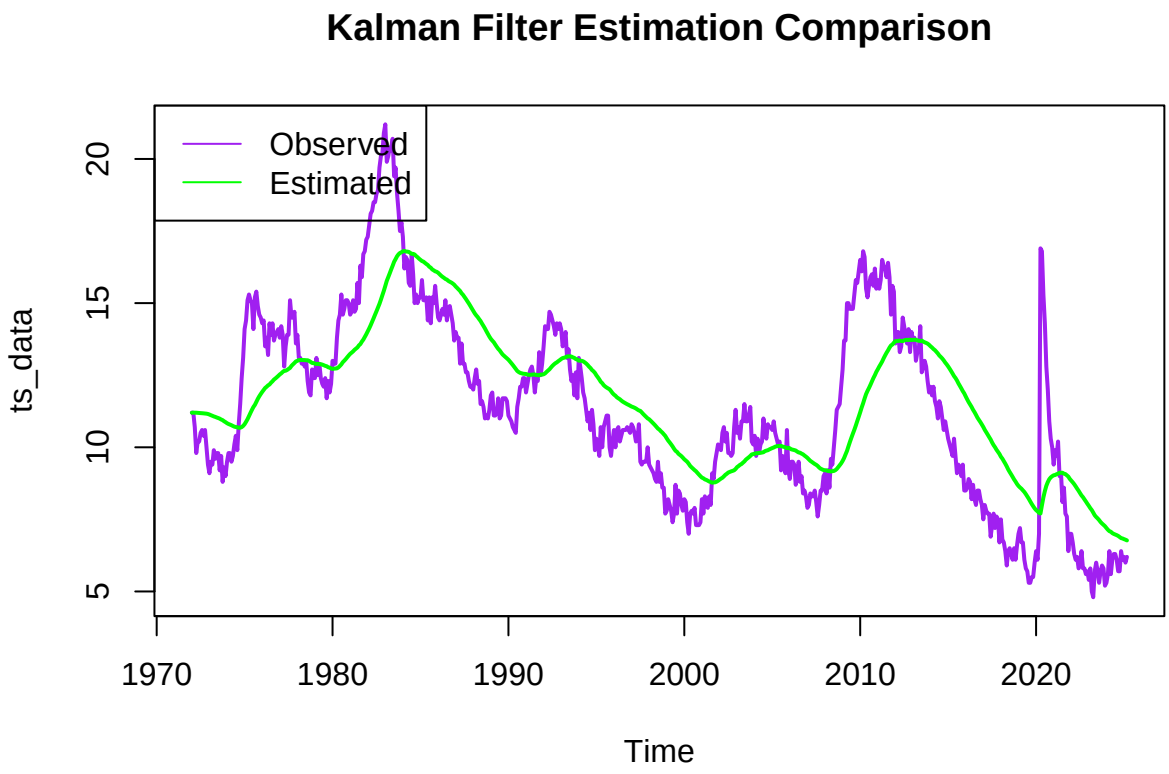
When q has a small value than what has been estimate:

```
# New initial conditions
ts_data <- ts(df$LNS14000006, start = c(1972, 1), frequency = 12)
n <- length(ts_data)
y <- as.numeric(ts_data)
sigma2_eta_com1 <- 0.01 # new variances
sigma2_epsilon_com1 <- 10

q <- sigma2_eta_com1/sigma2_epsilon_com1 # signal to noise ratio comparison

mu_pred <- numeric(n)  # mu_{t|t-1} comparison
P <- numeric(n)
v <- numeric(n)
K <- numeric(n)
F <- numeric(n)
```

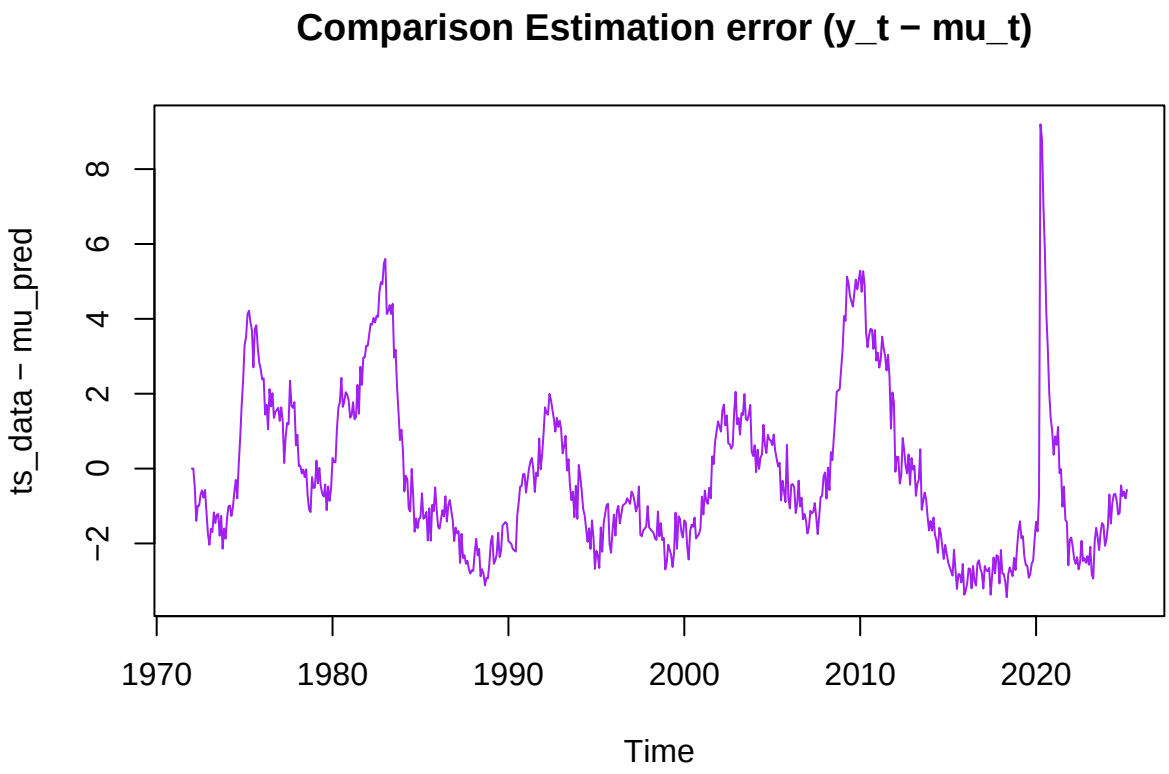
```r
llk <- numeric(n)

mu_pred[1] <- y[1]
P[1] <- q

# Iteration
for (t in 1:(n-1)) {
  # Prediction error
  v[t]<-y[t]-mu_pred[t]
  # Innovation variance
  F[t]<-P[t]+sigma2_epsilon_com1
  # Kalman gain
  K[t]<-P[t]/F[t]
  # Update step
  mu_pred[t+1]<-mu_pred[t]+K[t]*v[t]
  P[t+1]<-P[t]+sigma2_eta_com1-K[t]*F[t]*K[t]
  # Log-likelihood
  llk[t]<--0.5+log(F[t]+v[t]^2/F[t])
}

loglik <- sum(llk)
```

Now let's see the plots:

```r
ts.plot(ts_data, main = "Kalman Filter Estimation Comparison",
        col = "purple",type = "l", lwd = 2)
lines(ts(mu_pred, start = start(ts_data), frequency = frequency(ts_data)),
      col = "green",lwd=2)
legend("topleft", legend = c("Observed", "Estimated"),
       col = c("purple", "green"), lty = 1)
```




```
ts.plot(ts_data - mu_pred, col = "purple",
        main = " Comparison Estimation error (y_t - mu_t)")
```



As expected, if the value of q varies as described above, the Kalman Filter becomes smoother and captures the signal less accurately. As highlighted in the graph the shock encountered in 2020 can not be modeled also with a smoother version of the filter.

Note that we only analyze the case when q is smaller than the value we have obtained, since the Kalman Filter already uses a value, which is sufficient to perfectly capture the signal.